

TALLER QA-T01

FUNDAMENTOS DEL ASEGURAMIENTO DE LA CALIDAD

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

Resultado de aprendizaje: 593146

Duración: 4 horas

Modalidad: Trabajo colaborativo (equipos del proyecto)

Objetivo

Analizar los conceptos fundamentales del aseguramiento de la calidad del software y relacionarlos con el proyecto formativo, identificando oportunidades de mejora desde la perspectiva de un equipo de QA.

Contexto

Su equipo ha sido contratado como el nuevo **Departamento de Quality Assurance** de la empresa desarrolladora del proyecto que han construido desde los primeros trimestres.

Antes de iniciar el proceso de pruebas, el Director de Calidad solicita un diagnóstico conceptual para verificar que todo el equipo comprenda los principios fundamentales del aseguramiento de la calidad y pueda aplicarlos al proyecto.

Actividad 1. Analizando nuestro proyecto

Objetivo: Relacionar los conceptos de calidad con el proyecto desarrollado.

Completen la siguiente tabla utilizando ejemplos reales de su proyecto.

Concepto	Explique con sus palabras	Ejemplo de su proyecto
Calidad	Es lograr que el sistema sea confiable, rápido y seguro, cumpliendo las expectativas del negocio sin fallas.	Lograr que al ingresar el diagnóstico, el sistema calcule repuestos exactos y guarde los datos en MySQL sin que el backend en Node.js se caiga.
QA	Reglas, plantillas y estándares definidos antes y durante el desarrollo para prevenir defectos.	Creación rigurosa de plantillas de Épicas, Requisitos Funcionales (RF) y Casos de Uso antes de programar en React.

QC	Revisar el producto que ya construimos para atrapar defectos antes de que el código llegue a producción.	Revisar que la validación del middleware de Express bloquee a un 'Cliente' si intenta entrar a una ruta de 'Administrador'.
Testing	Ejecutar el software con diferentes datos para comprobar su comportamiento real.	Enviar peticiones POST al endpoint /api/login mediante Postman con contraseñas incorrectas (HTTP 401).
Verificación	Comprobar si construimos el sistema 'bien' técnicamente según arquitectura y diseño.	Revisar que el diagrama de base de datos MySQL cumpla con la normalización y llaves foráneas.
Validación	Comprobar si construimos el sistema 'correcto'. ¿Es esto lo que el negocio necesita?	Mostrarle la interfaz del 'Catálogo' al administrador en un sprint review para confirmar utilidad.

Regla: No se aceptan definiciones copiadas de Internet o de una IA. La explicación debe estar relacionada con el proyecto desarrollado por el equipo.

Actividad 2. ¿Dónde estamos hoy?

Analicen el estado actual de su proyecto y respondan.

Calidad

- ¿Qué características hacen que su software sea de calidad?
R/: La implementación de seguridad mediante Json Web Tokens (JWT), encriptado de contraseñas con bcrypt, arquitectura separando frontend (React/Vite) del backend (Node.js/Express) y documentación de la API con Swagger.
- ¿Qué aspectos consideran que aún deben mejorar?
R/: Optimizar tiempos de respuesta de consultas complejas en MySQL (rendimiento) y mejorar cobertura de excepciones en el lado del cliente (React) para errores amigables.

QA

- ¿Qué actividades de aseguramiento de la calidad realizaron durante el desarrollo del proyecto, aunque en ese momento no supieran que correspondían a QA?
R/: Diligenciamiento de la Matriz de Calidad ISO 25010, definición de Reglas de Negocio de seguridad y diseño de jerarquía documental (Épicas → RF → CU).

QC

4. ¿Qué actividades realizaron para verificar que el software funcionara correctamente?
R/: Evaluaciones manuales de los módulos desarrollados, revisión de código en pares y uso de Swagger UI para verificar endpoints.

Testing

5. ¿Qué tipos de pruebas han ejecutado hasta el momento?
R/: Pruebas unitarias en controladores principales (autenticación) y pruebas manuales exploratorias.
6. ¿Cuáles consideran que aún hacen falta?
R/: Pruebas de integración frontend-backend, pruebas de carga y pruebas UI automatizadas.

Verificación

7. ¿Cómo comprobaron que el software fue construido correctamente?
R/: Contrastando cada vista y controlador contra Criterios de Aceptación y Precondiciones definidos en los Requisitos Funcionales.

Validación

8. ¿Cómo comprobarán que el software realmente satisface las necesidades del cliente?
R/: Realizando pruebas de aceptación de usuario (UAT) donde el cliente interactúe con el módulo de reparaciones simulando un día normal en Celuaccel.

Actividad 3. Caso de análisis

Lean el siguiente caso.

El sistema presenta los siguientes problemas:

- Algunos usuarios no pueden iniciar sesión.
- Las consultas tardan más de un minuto.
- La aplicación solo funciona correctamente en Google Chrome.
- Los administradores pueden acceder a funciones restringidas para clientes.
- El cliente manifiesta que el sistema no cumple algunas necesidades del negocio.

Analicen el caso anterior y completen la siguiente tabla.

Situación	¿Corresponde a QA, QC o Testing?	Justifique su respuesta
-----------	----------------------------------	-------------------------

Login con errores	Testing	Es el resultado de la ejecución de una prueba sobre la función de inicio de sesión evidenciando falla técnica.
Lentitud del sistema	QC	Se está evaluando el producto contra el estándar de rendimiento (ej. tiempo < 2 segundos).
Incompatibilidad con navegadores	QC	Falla detectada al inspeccionar el producto final contra la matriz de compatibilidad definida en QA.
Problemas de permisos	Testing / QC	Defecto técnico (control de acceso) detectado al ejecutar escenarios de prueba de roles.
Requisitos del cliente no satisfechos	Falla de QA / Validación	Ocurrió porque desde el inicio (QA) no se definió bien el alcance, y no se validó continuamente.

Actividad 4. Verificación vs Validación

Para cada situación indique si corresponde a **Verificación**, **Validación** o **Ambas**.

Situación	Verificación	Validación	Justificación
Revisar el documento de requisitos antes de programar	X		Comprueba que el documento (RF, CU) esté construido

			técnica y lógicamente bien.
Ejecutar pruebas funcionales del Login	X		Comprueba que el código cumple con las especificaciones técnicas creadas por los desarrolladores.
El cliente prueba el sistema y confirma que satisface sus necesidades		X	Confirma que el producto final tiene valor real para el negocio, independiente de la codificación.
Revisar el código fuente mediante Pull Request	X		Evalúa buenas prácticas en React/Node.js y que no existan errores de sintaxis.
Comparar el sistema con los criterios de aceptación	X		Comprueba el producto contra reglas técnicas (Ej. clave cifrada con bcrypt).
Revisar el diseño de la base de datos	X		Asegura que el modelo entidad-relación en MySQL cumpla las formas normales.

Actividad 5. Costos de la no calidad

Imaginen que el proyecto ya fue entregado al cliente.

Ahora aparecen los siguientes problemas.

Problema	Consecuencia para el cliente	Consecuencia para la empresa	¿Cómo se habría evitado?
El Login deja de funcionar en producción	Bloqueo total de la operación (no puede registrar ingresos).	Pérdida de credibilidad, soporte de emergencia.	Aplicando Testing automatizado de regresión.
Se pierden datos de los usuarios	Pérdida de historiales, equipos y facturación.	Demandas legales, daño irreparable de imagen.	QA definiendo políticas de backups en MySQL.
El sistema es muy lento	Frustración de técnicos, demoras en atención.	Quejas constantes y riesgo de cancelación.	QC enfocado en pruebas de carga y estrés.
La aplicación presenta fallos de seguridad	Robo de información, suplantación de clientes.	Sanciones, multas, quiebra del proyecto.	Auditorías de seguridad, pruebas de penetración.
El cliente rechaza el producto	Tiempo y dinero invertidos sin solución real.	Pérdida financiera total del proyecto.	Con Validaciones continuas en cada sprint.

Finalmente respondan:

- ¿Cuál de estos problemas sería el más costoso para su proyecto?
R/: La pérdida de datos y fallos de seguridad. Perder el historial de a quién pertenece un dispositivo (Celucel) causa problemas legales graves.
- ¿Qué impacto tendría en la imagen de la empresa?
R/: Destruiría completamente la confianza. Un sistema de servicio técnico debe ser altamente fiable.
- ¿Qué actividades de QA podrían prevenir estos problemas?
R/: Matrices de riesgo detalladas, estructurar matriz ISO 25010 estricta para fiabilidad y exigir revisiones de código enfocadas en vulnerabilidades.

Actividad 6. Aplicación al proyecto

Reúnanse como equipo y elaboren una lista de actividades de calidad que realizarán durante este trimestre.

Completen la siguiente tabla.

Actividad QA	Responsable	Evidencia
Elaborar Plan Maestro de Pruebas	Joya Peña	Documento Plan Maestro de Pruebas.
Diseñar Casos de Prueba	Parra Pinto	Matriz de Casos de Prueba (RF).
Ejecutar pruebas	Chaves Ravelo	Reporte de ejecución en Postman/Cypress.
Registrar defectos	Todo el equipo	Tablero de Trello / Matriz de Bugs.
Elaborar informe de pruebas	Joya Peña / Parra Pinto	Documento final de métricas.
Proponer mejoras	Todo el equipo	Acta de reunión y refactorización del código.

Pueden agregar otras actividades según su proyecto.

Actividad 7. Mapa conceptual

Con base en el análisis realizado durante el taller, elaboren un **mapa conceptual** donde relacionen los siguientes conceptos:

- Calidad.
- QA.
- QC.
- Testing.
- Verificación.
- Validación.
- Mejora Continua.

El mapa debe incluir **ejemplos del proyecto**, no únicamente definiciones.

Herramientas sugeridas:

- Draw.io

- Canva
- Lucidchart
- Miro
- PowerPoint

(Estructura para el Mapa Conceptual sugerida en base a Celuaccel)

- Nodo Central: CALIDAD DE SOFTWARE

- Rama 1: Aseguramiento (QA) -> Prevención -> Ej: Matriz ISO 25010 y definir bcrypt.

- Rama 2: Control (QC) -> Detección -> Testing -> Ej: Ejecutar Postman sobre la API.

- Rama 3: Métodos -> Verificación (Código consume endpoints) y Validación (Vista Catálogo es útil).

- Rama 4: Mejora Continua -> Ej: Refactorizar Node.js si notamos lentitud.

Actividad 8. Cuadro comparativo

Elaboren un cuadro comparativo como el siguiente.

Concepto	Objetivo	¿Cuándo se aplica?	Responsable	Ejemplo del proyecto
Calidad	Entregar valor y satisfacción total.	En todo el ciclo de vida.	Todo el equipo.	Sistema estable, seguro y que gestiona reparaciones.
QA	Prevenir defectos mejorando procesos.	Antes y durante el diseño.	Analistas / QA.	Definir documentación en Swagger y formato RF.
QC	Identificar defectos en el producto.	Durante la construcción.	Equipo de Pruebas / QC.	Revisar que la web React no permita campos vacíos.
Testing	Comprobar comportamiento o ejecutando código.	En fase de pruebas.	Testers.	Inyecciones SQL en login o cargas en Express.
Verificación	Asegurar cumplimiento técnico (Hacerlo bien).	Durante el desarrollo.	Desarrolladores y QA.	Revisar que BD MySQL tenga llaves foráneas correctas.

Validación	Asegurar necesidades del negocio (Hacer lo correcto).	Entregas y demostraciones.	Cliente, Product Owner.	Mostrar flujo de Orden de Servicio al cliente.
------------	---	----------------------------	-------------------------	--

Evidencias a entregar

Cada equipo deberá entregar:

- Taller completamente diligenciado.
- Mapa conceptual.
- Cuadro comparativo.
- Conclusiones del equipo.

Criterios de evaluación

Criterio	Porcentaje
Análisis y aplicación de los conceptos	30%
Relación con el proyecto formativo	25%
Calidad del mapa conceptual	15%
Calidad del cuadro comparativo	15%
Argumentación y conclusiones	15%